
FAHIM: Framework for Arabic Human-centered Interaction on Mobile

Nawaf bin Jurayyan, Ammar Abdulrahman, Khaled Alshahry,
Abdulaziz Batis, Muhammad Mubashar

nawaf.binjurayyan@kaust.edu.sa, ammar.abdulrahman@kaust.edu.sa, khaled.alshahry@kaust.edu.sa,
abdulaziz.batis@kaust.edu.sa, muhammad.mubashar@strath.ac.uk

Abstract

We present FAHIM

📅 **Date:** July 25, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Key Assumptions and Scope	2
1.3	Proposed Solution	2
2	Related Work	2
2.1	TalkBack	3
2.2	Insight	3
3	Methodology	3
3.1	System Overview	3
3.2	System Architecture	4
3.3	Speech Understanding Module	4
3.4	UI State Analyzer	4
3.5	Task Planning Module	5
3.6	Action Execution Module	5
3.7	Feedback and Safety Module	5
3.8	Evaluation Procedure	5
4	Results	5
5	Discussion	5
6	Limitations and Future Work	6
7	Conclusion	6
	References	6

1 Introduction

In an increasingly digital world, smartphones and mobile applications have become essential tools for daily life, supporting activities ranging from communication to banking and healthcare. However, for visually impaired and blind individuals, navigating complex mobile user interfaces (UIs) remains a major barrier to digital independence. While traditional screen readers such as Apple's VoiceOver and Android's TalkBack provide spoken feedback, they still require users to manually navigate interface elements one step at a time, making complex tasks time-consuming and cognitively demanding.

1.1 Problem Statement

The core problem is that existing mobile accessibility tools are passive; they read what is on the screen but require the user to perform manual, step-by-step navigation. For a visually impaired user, completing even a simple task on an unfamiliar mobile application can require considerable time and effort. There is a critical need for an active, intelligent assistant that can interpret high-level spoken commands, navigate mobile interfaces autonomously on behalf of the user, and verbally report the actions taken and their outcomes.

1.2 Key Assumptions and Scope

In developing this system, we work under the following assumptions:

- The target user interacts with the application exclusively using natural language voice commands.
- The system operates on Android devices using the Android Accessibility framework.
- The system relies on an on-device or lightweight Large Language Model (LLM) capable of translating user requests into concrete UI actions (such as clicking, typing, or scrolling) while reasoning over the current accessibility tree.

1.3 Proposed Solution

To address this challenge, we introduce **FAHIM**, an intelligent AI-agent assistant designed to enable hands-free, voice-driven interaction with Android applications for blind individuals.

Our system acts as a smart intermediary between the user and the mobile interface. Instead of manually navigating the screen, users simply describe the task they wish to accomplish (e.g., "*Call Khalid*" or "*Turn on Wi-Fi*"). The assistant interprets the spoken request, determines the required sequence of UI actions, executes them autonomously using Android Accessibility Services, and provides spoken feedback describing the actions performed and their outcomes. By transforming traditional screen navigation into a natural conversational interaction, FAHIM aims to improve digital accessibility, efficiency, and user independence.

2 Related Work

Recent advances in mobile accessibility have demonstrated the potential of conversational interfaces and Large Language Models (LLMs) to improve the interaction experience of blind and visually impaired users. Existing accessibility solutions such as TalkBack provide fundamental screen reading capabilities, while recent LLM-based systems aim to enable more natural and intelligent interaction. This section reviews the conventional TalkBack screen reader and the recently proposed Insight system.

2.1 TalkBack

TalkBack is the default Android screen reader that enables blind and visually impaired users to interact with mobile devices through spoken feedback and gesture-based navigation. It provides auditory descriptions of interface elements as users navigate the screen using swipe and touch gestures. In addition, TalkBack supports accessibility features such as voice commands, braille displays, and customizable verbosity settings.

Despite its widespread adoption, TalkBack follows a sequential navigation paradigm in which users must manually traverse interface elements one at a time. This interaction style often increases cognitive load, especially when completing complex multi-step tasks, since users must remember screen layouts and navigate through multiple interface components before reaching the desired action. Furthermore, TalkBack provides limited understanding of the overall screen context, making it difficult to efficiently perform goal-oriented interactions or obtain concise summaries of complex interfaces.

2.2 Insight

Insight is a recently proposed Android accessibility service that leverages Large Language Models (LLMs) to provide conversational interaction for blind and visually impaired users. Unlike traditional screen readers, Insight combines natural language dialogue with real-time screen summarization, allowing users to express high-level goals rather than manually navigating every interface element. The system analyzes the Android accessibility hierarchy, generates contextual summaries of the current screen, answers user questions, and produces executable UI actions through an LLM-powered planning process [1].

User studies reported that Insight reduced task completion time and cognitive effort while improving perceived usability compared with TalkBack. Participants preferred the conversational interaction model because it enabled more natural and efficient interaction. However, the study also identified several limitations, including reliance on cloud-hosted LLMs, privacy concerns associated with transmitting screen information, limited interruption during long responses, and the need for more robust conversational interaction [1].

Compared with existing approaches, the proposed FAHIM framework focuses on on-device execution and Arabic language support to improve privacy and better serve Arabic-speaking blind users. In addition, the system incorporates explicit user confirmation before performing sensitive actions, ensuring safer and more reliable task execution.

3 Methodology

This chapter presents the proposed framework, FAHIM, including its overall architecture, system components, and evaluation methodology.

3.1 System Overview

FAHIM (Framework for Arabic Human-centered Interaction on Mobile) is an on-device intelligent mobile agent designed to assist Arabic-speaking blind users in completing multi-step tasks through natural voice interaction. Rather than requiring users to navigate graphical interfaces manually, the framework allows users to express their desired task using spoken Arabic commands. The system then interprets the user's intent, analyzes the current mobile interface, plans the required sequence of actions, and executes them autonomously while providing continuous spoken feedback.

As an initial proof of concept, the proposed framework focuses on two Android applications:

the **Settings** application and the **Phone** application. These applications were selected because they represent common daily tasks while providing different interaction challenges. The Settings application evaluates hierarchical navigation and configuration tasks, whereas the Phone application evaluates goal-directed interactions such as searching contacts and initiating phone calls.

3.2 System Architecture

The proposed FAHIM framework consists of the following five main components:

1. Speech Understanding Module
2. UI State Analyzer
3. Task Planning Module
4. Action Execution Module
5. Feedback and Safety Module

These components interact sequentially to complete the user's requested task. The overall system workflow is summarized as follows:

1. The user provides a task using natural Arabic speech.
2. The Speech Understanding Module transcribes the speech and identifies the user's intended goal.
3. The UI State Analyzer extracts the current Android Accessibility Tree and represents the current interface state.
4. The Task Planning Module determines the next action based on the user goal and the current UI state.
5. The Action Execution Module performs the selected action using Android Accessibility APIs.
6. The Feedback and Safety Module provides spoken feedback and requests confirmation before executing sensitive actions.
7. The process repeats from Step 3 until the requested task is completed or no valid action remains.

3.3 Speech Understanding Module

The Speech Understanding Module serves as the entry point of the FAHIM framework by converting the user's spoken Arabic command into text. To perform speech recognition, the framework utilizes Android's built-in Speech-to-Text (STT) service, enabling seamless integration with the Android platform while supporting natural voice interaction.

The transcribed text is then forwarded to the Qwen3 Large Language Model (LLM), where it is combined with the current interface state to generate the next accessibility action required to accomplish the user's request. This allows users to express tasks using natural language without relying on predefined command templates.

3.4 UI State Analyzer

The UI State Analyzer retrieves the Android Accessibility Tree through Android Accessibility Services. The extracted hierarchy contains information about visible interface components, including buttons, text fields, switches, menus, and their accessibility labels. This information is transformed into a structured representation describing the current application state, which is subsequently used by the planner during decision making.

3.5 Task Planning Module

The Task Planning Module is responsible for determining the sequence of actions required to accomplish the user's requested task. Using the Qwen3 Large Language Model (LLM), the module reasons over the transcribed user request together with the current Android Accessibility Tree to generate the most appropriate next accessibility action.

After each action is executed, the updated interface state is retrieved by the UI State Analyzer and provided to the planner. The planning process is repeated until the requested task is successfully completed or no valid action remains. This iterative reasoning mechanism enables the framework to adapt dynamically to changes in the user interface while executing multi-step tasks.

3.6 Action Execution Module

The Action Execution Module translates the planner's decisions into executable Android interactions using the Accessibility API. Supported operations include clicking interface elements, scrolling, entering text, navigating between screens, and performing other accessibility-supported actions. After execution, the updated interface state is passed back to the planner for the next planning iteration.

3.7 Feedback and Safety Module

To improve usability and maintain user trust, the framework continuously provides spoken feedback describing its current progress. For operations that may have significant consequences, such as placing a phone call or modifying important system settings, the framework requests explicit user confirmation before executing the corresponding action. This human-centered interaction mechanism ensures that users remain in control throughout the execution process.

3.8 Evaluation Procedure

4 Results

This section should include:

- Quantitative results and plots. You should define a baseline to identify whether the found values are good or bad.
- Comparison with expectations or benchmarks.
- Component-level results or ablations where applicable.

Table 1 Example performance table. Report a baseline so the reader can judge whether your numbers are good.

Component	Metric	Baseline
Module A	95%	90%
Module B	88%	85%
Full system	91%	87%

5 Discussion

Mention the results in the Results section above, THEN interpret them here, do not just repeat them.

- Discuss the results reported in the previous section. Particularly: Why do the results look the way they do? Where do they meet or miss your expectations and the benchmarks?
- Unexpected findings or discoveries.

6 Limitations and Future Work

State your system's limitations HONESTLY. For each limitation, propose a concrete next step or improvement.

- Current limitations, e.g., dataset size, compute budget, robustness, or scope,
- Concrete next steps to overcome each limitation.
- Promising directions for extending the project.

7 Conclusion

Summarize in one paragraph the most important points of your project and highlight the key findings, e.g., our system's accuracy outperformed the commonly available commercial products by XX%. Close with the main takeaway a reader should remember.

References

- [1] Yin-Chen N. Chen et al. "Insight: An LLM-Based Conversational Mobile Assistant for Blind Users". In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 10.1 (2026). DOI: [10.1145/3746044](https://doi.org/10.1145/3746044).